

CintOS – справочник API ядра

Системные вызовы, capability-модель, формат исполняемых файлов и crt0. Версия соответствует workspace kernel_base 0.1 / kernel_x86 0.1 / cintos-user 0.1.

1. Архитектура

Ядро разделено на архитектурно-независимый слой kernel_base (capability-менеджер, планировочные интерфейсы, трейты памяти/IOMMU/прерываний, ехес-слой), платформенный kernel_x86 (страничные таблицы x86_64, драйверы VT-d/AMD-Vi, ACPI, загрузчик образов, entry-регистрация) и userspace-библиотеку cintos-user (ABI-обёртки, crt0). Все ресурсы доступны задачам только через capability; групповые права неймспейса являются потолком для прав отдельных потоков.

2. Конвенция системного вызова (x86_64)

Элемент	Значение
Номер сисколла	RAX
Аргументы 1..6	RDI, RSI, RDX, R10, R8, R9 (порядок SysV; R10 вместо RCX)
Возврат	RAX: 0/результат – успех; старший бит (0x8000_0000_0000_0000) – ошибка
Портится	RCX, R11 (железо SYSCALL), RAX
Инструкция	SYSCALL; возврат – SYSRET (entry-стаб порта)
Успех с результатом	например, id созданной capability; id 0 валиден – различение по старшему биту

Коды ошибок: 1 NO_CURRENT_TASK, 2 RIGHTS_DENIED (отказ групповых прав неймспейса), 3 NOT_FOUND, 4 SLOT_OCCUPIED, 5 SLOT_EMPTY, 6 CAP_REVOKED, 7 RIGHTS_EXCEEDED, 8 SLAB, 9 QUOTA, 10 IDS_EXHAUSTED, 11 INVALID_ARG, 12 INTERNAL (константы в traits::syscall::syscall_result и cintos-user::abi::result).

3. Таблица системных вызовов (плоская нумерация)

NR	Имя	Аргументы (по порядку)	Возврат/семантика
0	SchedYield	–	добровольная передача ядра
1	SchedRegisterTask	entry, code_size, stack_size, task_cap	регистрация runtime-метаданных; ок – задача готова к запуску
2	SchedDestroyTask	task_cap	уничтожение задачи; для self-exit задача передаёт собственный id (crt0::exit)
3	SchedBlockOnObject	object_id	сон на объекте ожидания (OneShot)
4	SchedReleaseObject	object_id	пробуждение ожидающих объект
5..8	Memory: Extend/Decrease/MountCap/UnmountCap	–	зарезервировано (заглушки)
10/11	IpcSend/IpcWait	cap_id, msg_ptr, msg_size cap_id, tgt_ptr, max	транспорт сообщений (заглушки); пересылка капабилити – NR 24
16	CapCreateNamespace	dst_slot, max_tasks, max_mem, badge, rights	создание неймспейса; возвращает id capability; требует CAP_MANAGE
17	CapCreateIpcPool	owner_task, dst_slot	capability на пул IPC-памяти; CAP_MANAGE MEMORY_ALLOC
18	CapCreateMmio	owner_task, dst_slot, phys, pages	capability на MMIO-регион; CAP_MANAGE MMIO_MAP
19	CapCreateIrq	owner_task, dst_slot, cpu, vector	capability на IRQ; CAP_MANAGE IRQ_BIND

NR	Имя	Аргументы (по порядку)	Возврат/семантика
20	CapMint	src_task, src_slot, dst_task, dst_slot, rights	производная копия (права $\subseteq$ источника); требует CAP_MINT + право Mint
21	CapClone	src_task, src_slot, dst_task, dst_slot	копия в той же мембране; требует право Clone
22	CapRevoke	task, slot	ревок мембраны слота – протухают все производные
23	CapDestroy	task, slot	ревок + снятие записи
24	CapTransfer	sender, receiver, src_slot, dst_slot, rights	пересылка capability через IPC; обе стороны проверяются
28	IrqWait	lines_mask, mask_ptr, mask_slots	сон до срабатывания линий; ядро пишет [count][line0..] в userspace-массив; требует IRQ_BIND
32	IommuCreateDomain	dst_slot	DMA-домен; требует DMA_ATTACH; возвращает id capability
33	IommuAttachDevice	task, slot, bus, dev, func	присоединение PCIe-устройства (BDF от userspace)
34	IommuMapDma	task, slot, iova, phys, pages, prot	DMA-маппинг (second-stage)
35	IommuUnmapDma	task, slot, iova, pages	снятие DMA-маппинга
36	IommuCreatePasidSpace	domain_task, domain_slot, mode, dst_slot	PASID-пространство; mode: 0=Dedicated, 1=OwnAddressSpace (SVA)
37	IommuAttachPasid	domain_task/slot, space_task/slot, BDF	привязка (device, PASID) -> first-stage AS
38	IommuDetachPasid	domain_task/slot, space_task/slot	отвязка устройства
39	IommuMapVa	space_task, slot, gva, phys, pages, prot	маппинг first-stage (dedicated-режим)
40	IommuUnmapVa	space_task, slot, gva, pages	снятие first-stage
41	IommuDestroyPasidSpace	space_task, slot	уничтожение (авто-отвязка)

4. Модель прав

Высокогранулярные права capability (биты маски в CapMint/CapTransfer): Clone=1, Mint=2, Send=4. Групповые права неймспейса (битмаска rights в CapCreateNamespace) – потолок для всех потоков группы: TASK_CREATE=1, MEMORY_ALLOC=2, MMIO_MAP=4, IRQ_BIND=8, IPC_SEND=16, CAP_TRANSFER=32, CAP_MINT=64, CAP_MANAGE=128, DMA_ATTACH=256. Отказ неймспейса имеет приоритет: даже корректная capability не даст доступ, если группового права нет.

5. Формат исполняемых файлов и crt0

Образы системных серверов – ELF64 с EI_OSABI = 0xC1 (CintOS), ET_EXEC, e_machine = EM_X86_64 (62). Обычные Linux-ELF отклоняются гейтом OS ABI. Формат подключаем: трейт ExecFormat (probe/parse) в kernel_base::exec; нейтральное ImageInfo (entry + сегменты vaddr/file/flags) – загрузчику arch-слоя неважно, что внутри. Новый формат = регистрация ещё одного ExecFormat в реестре.

Раскладка стартового стека (начальный RSP указывает на argc, кратен 16):

```

stack_top (высокие адреса)
  строки argv/envp (NUL-терминированные)
  auxv: (tag,val)*16B ... AT_NULL
  envp: указатели, NULL
  argv: указатели, NULL
  argc: u64                <- начальный RSP (кратен 16)

```

AUXV: 7=PAGESZ, 9=ENTRY, 0xC170_0001=SELF_CAP (id TaskTCB), 0xC170_0002=NS_CAP (id корневой capability неймспейса). Bootstrap-слоты cspace: 0 = self-TCB (полные права), 1 =

корневой неймспейс.

crt0 (cintos_user::crt0): _start читает стек, публикует args()/bootstrap(), вызывает main(argc, argv, envp) бинаря; возврат из main = self-exit (SCHED_DESTROY_TASK со своим cap). Паник-хендлер – аварийный self-exit.

6. Доступ к ресурсам из userspace

Операция	Как
Узнать себя	crt0::bootstrap() -> { self_cap, namespace_cap, page_size, entry }
Выход	вернуть код из main (crt0 делает self-exit)
Создать группу задач	CapCreateNamespace; права – битмаска групповых прав
Создать задачу	kernel-internal на старте; syscall в roadmap
Память под IPC	CapCreateIpcPool + CapTransfer получателю
Отдать capability	CapTransfer (проверки обеих сторон: Send у отправителя, право класса ресурса у получателя)
Драйвер устройства	CapCreateMmio (ядро) + IOMMU-инвокации 32..41 (устройство к домену, DMA-маппинг)
SVA (свои указатели в DMA)	IommuCreatePasidSpace mode=1 (OwnAddressSpace) + AttachPasid; first-stage root = CR3 процесса
Ждать IRQ	IrqWait(lines_mask, массив): после пробуждения массив содержит count и номера линий

7. Ограничения текущей версии

Мемору-домен (5..8) и IPC-транспорт (10/11) – заглушки; entry-стаб SYSCALL/SYSRET и планировщик живут в порту; код возврата main не сохраняется при self-exit; PASID ограничен 512 на юнит; Intel QI и AMD GN-инвалидации требуют верификации на QEMU. Дорожная карта: TaskCreate/MemMap сисколлы, таймауты ожидания, Endpoint-capability, фолт-доставка супервизору.